Detección de Malware en Aplicaciones de Android

Juan Diego Sanchez
jd.sanchez2@uniandes.edu.co

Estefanía Laverde Becerra
e.laverdeb@uniandes.edu.co

Luis Fajardo Bravo
lf.fajardo@uniandes.edu.co

Jorge Bustamante
j.bustamantep@uniandes.edu.co

1 Planteamiento del Problema y Motivación

En la actualidad, el sistema operativo Android cuenta con 3.9 mil millones de usuarios BankMyCell [2026] y una participación de mercado superior al 70%. Esta adopción masiva, sumada a la flexibilidad de su ecosistema y la apertura de sus mercados de aplicaciones, ha generado un entorno propenso a la proliferación de software malicioso (*malware*). Agentes malintencionados aprovechan estas características para distribuir aplicaciones diseñadas para comprometer la integridad de los datos y la privacidad del usuario a escala global.

Históricamente, la detección de estas amenazas se ha basado en dos metodologías principales: el **análisis mediante firmas**, que utiliza funciones *hash* para identificar archivos idénticos a muestras conocidas, y los **sistemas basados en reglas**, que emplean heurísticas para detectar comportamientos sospechosos.

Sin embargo, el panorama actual de amenazas ha invalidado estos enfoques tradicionales debido al uso de técnicas de evasión avanzadas:

- **Ofuscación:** Transformación del código fuente para volverlo ininteligible sin alterar su lógica funcional.
- **Polimorfismo:** Mutación de la apariencia binaria del software en cada iteración o descarga.
- **Empaquetamiento (*packing*):** Cifrado o compresión del ejecutable original para ocultarlo tras una capa externa.

Estas técnicas permiten alterar la estructura del código sin modificar su comportamiento malicioso, logrando que el *malware* pase desapercibido ante los métodos de detección convencionales.

2 Metodología Propuesta

Ante este escenario, el presente proyecto propone el desarrollo de un sistema de clasificación de aplicaciones Android basado en **análisis estático multimodal**. A diferencia del análisis dinámico, este enfoque permite inspeccionar las propiedades del software sin necesidad de ejecutarlo, eliminando el riesgo de infección durante el proceso.

El objetivo principal es la categorización de las aplicaciones en cinco clases:

- **Adware:** Material publicitario oculto en aplicaciones infectadas que ejecuta procesos persistentes de despliegue de anuncios y robo de información.
- **Banking:** Troyanos que imitan interfaces bancarias para capturar credenciales y enviarlas a servidores de Comando y Control (C&C).
- **SMS Malware:** Explotación del servicio de mensajería para interceptar contenidos y coordinar ataques mediante servidores C&C.
- **Riskware:** Programas legítimos explotables que pueden escalar a otras formas de *malware* mediante la instalación de nuevos componentes infectados.
- **Benign:** Aplicaciones validadas (vía VirusTotal) que no presentan comportamientos maliciosos.

La motivación técnica del enfoque multimodal radica en que una sola fuente de información puede llegar a ser insuficiente para capturar la complejidad del *malware* actual. Por ello, el modelo integrará datos tabulares, imagenes provenientes del binario del archivos ejecutable de la aplicación y secuencias API calls, definidas a profundidad en la sección Dataset 4.

3 Estado del Arte

3.1 Multimodal Deep Learning for Android Malware Classification. Arrowsmith et al. [2025]

El trabajo "Multimodal Deep Learning for Android Malware Classification" propone un enfoque multimodal que combina dos representaciones estáticas extraídas de los archivos DEX: imágenes binarias RGB generadas a partir del bytecode y grafos de llamadas a funciones (FCG), los cuales modelan las interacciones entre métodos. Para ello se utiliza una estrategia de late fusion, donde una

CNN procesa las imágenes y una GNN analiza los grafos; posteriormente, las probabilidades de ambos modelos se combinan mediante un perceptrón multicapa (MLP) que actúa como clasificador final.

Los resultados experimentales, obtenidos a partir de un dataset multimodal de 5000 aplicaciones derivado de MalNet y MalNet-Tiny, muestran que el modelo multimodal supera a los modelos unimodales. Por ejemplo, una CNN simple alcanza aproximadamente 83.1% de accuracy y una GNN basada en GCN alrededor de 80.6%, mientras que la combinación multimodal mediante late fusion logra hasta 90.6% de accuracy, precision, recall y F1-score en el modelo DenseNet121 + GIN con meta-clasificador optimizado.

Estos resultados evidencian que la integración de representaciones heterogéneas (patrones visuales del bytecode y relaciones estructurales del código) mejora significativamente la capacidad de clasificación y la robustez del sistema, posicionando la late fusion como una estrategia eficiente para la detección multimodal de malware.

3.2 DMLDroid: Deep multimodal fusion framework for Android malware detection with resilience to code obfuscation and adversarial perturbations. Trung et al. [2025]

Por la parte de la investigación realizada para el trabajo DMLDroid propone un sistema avanzado de detección de malware en Android basado en aprendizaje profundo multimodal, diseñado específicamente para mejorar la robustez frente a técnicas de evasión como la ofuscación de código y los ataques adversarios. El marco integra tres modalidades complementarias de información extraídas de aplicaciones Android: características tabulares derivadas del análisis del archivo AndroidManifest.xml, representaciones visuales RGB generadas a partir de los archivos DEX, que contienen el bytecode compilado de la aplicación, y secuencias derivadas de grafos de llamadas API, que modelan las dependencias estructurales entre métodos dentro del programa.

Cada modalidad es procesada mediante arquitecturas especializadas de aprendizaje profundo: un MLP para las características tabulares, una CNN para las representaciones visuales derivadas de los archivos DEX y un modelo basado en DistilBERT para capturar dependencias semánticas en las secuencias de llamadas a APIs extraídas de grafos. Posteriormente, las representaciones latentes obtenidas de cada modalidad se integran mediante diferentes estrategias de fusión multimodal, entre las que se incluyen concatenación simple, self-attention, cross-attention, gated fusion y dynamic weighted fusion, lo que permite evaluar de manera sistemática el impacto de cada técnica en el desempeño del sistema. Los experimentos realizados sobre el dataset CICMalDroid 2020 muestran que el enfoque multimodal supera claramente a los modelos unimodales, alcanzando 97.98% de accuracy y 98.67% de F1-score, y manteniendo además una precisión superior al 98% incluso bajo escenarios de ofuscación de código y generación de ejemplos adversarios, lo que evidencia que la combinación de múltiples fuentes de información mejora la capacidad de generalización del modelo frente a variantes de malware. Este trabajo se relaciona directamente con el presente proyecto, ya que respalda la hipótesis de que la detección efectiva de malware requiere integrar representaciones heterogéneas del software para capturar tanto patrones estructurales como comportamentales; en particular, valida empíricamente el uso conjunto de características estáticas tabulares y representaciones visuales del bytecode, enfoque que también constituye la base metodológica de este estudio para clasificar aplicaciones Android en categorías como Adware, Banking, SMS malware y Riskware, mientras que la arquitectura de fusión multimodal propuesta en DMLDroid proporciona una referencia metodológica relevante para el diseño de sistemas de clasificación robustos frente a técnicas modernas de evasión.

3.3 Android Malware Detection Based on RGB Images and Multi-feature Fusion. Wang et al. [2024]

El estudio Android Malware Detection Based on RGB Images and Multi-feature Fusion propone un enfoque de detección basado en la representación visual del software, donde múltiples componentes internos de una aplicación Android se transforman en imágenes para su posterior análisis mediante modelos de aprendizaje profundo. En particular, el método extrae tres fuentes principales de información de los archivos APK: el archivo DEX (que contiene el código ejecutable de la aplicación), el archivo AndroidManifest.xml (que define permisos, componentes y configuraciones de la aplicación) y las llamadas a APIs del sistema.

Cada uno de estos elementos se convierte inicialmente en imágenes en escala de grises, interpretando directamente los bytes del archivo como valores de intensidad de píxel. Posteriormente, se aplican técnicas de mejoramiento de características visuales, como Canny edge detection, histogram equalization y adaptive thresholding, con el objetivo de resaltar patrones estructurales relevantes dentro de los datos binarios. Finalmente, las tres imágenes resultantes se combinan en una imagen RGB multicanal, donde cada canal representa una modalidad distinta de información del software analizado.

Estas imágenes RGB son utilizadas como entrada para distintos modelos de clasificación de imágenes basados en redes neuronales profundas, incluyendo arquitecturas como AlexNet, ResNet, GoogleNet y MobileNetV2, permitiendo que la red neuronal aprenda automáticamente patrones espaciales asociados con diferentes familias de malware. Los resultados experimentales muestran que este enfoque alcanza una precisión de hasta 97.25% superando a métodos que utilizan únicamente representaciones derivadas del archivo DEX, lo que demuestra que la combinación de múltiples fuentes de información mejora significativamente la capacidad de discriminación del modelo.

4 Dataset

El dataset que será usado corresponde al conjunto *Multimodal CIC AndMal2020*, disponible públicamente en Kaggle [2020] descrito en Trung et al. [2025]. Esta base de datos es una evolución de CICMalDroid 2020, [Mahdavifar et al., 2020, 2021], diseñada específicamente para soportar modelos que integran múltiples fuentes de información, como tabular, textual e imagenes.

El conjunto de datos se organiza en tres modalidades principales que permiten un análisis estático exhaustivo:

- **Tabular (Características estáticas):** Consiste en un conjunto de características numéricas extraídas directamente de los archivos *.APK*. Estos datos representan principalmente el conteo de frecuencias de llamadas a funciones críticas, métodos del sistema y permisos solicitados por la aplicación (como acceso a contactos, cámara o servicios de red).
- **Imagenes RGB:** Son representaciones visuales generadas a partir de los archivos *.DEX* (Dalvik Executable), los cuales contienen el código compilado que Android ejecuta. Para formar estas imágenes, el flujo de bytes binarios del archivo se mapea a canales de color (RGB). La estructura del código se traduce en texturas y patrones visuales específicos.
- **Secuencia (secuencias de llamadas a API):** Se presenta como cadenas de texto que contienen secuencias cronológicas de llamadas a APIs. Estos datos permiten capturar la lógica del flujo de control de la aplicación, identificando patrones de ejecución que podrían ser característicos de software malicioso.

Todas las muestras en el dataset están debidamente etiquetadas en las cinco categorías principales anteriormente mencionadas: *Adware*, *Banking*, *SMS Malware*, *Riskware* y *Benign*.

5 Exploración de los Datos

5.1 Datos Tabulares

Se cuenta con un total de **16691** aplicaciones de Android, cada una descrita mediante **402 características** estáticas. En este conjunto, la columna *apk_name* cumple como identificador único de cada muestra, mientras que la columna *Class* define la categoría de malware asignada.

En términos de la distribución de clases, se observa un notable **desbalanceo de datos**, como se ilustra en la Figura 1. La categoría *Adware* presenta un número de muestras significativamente menor en comparación con clases dominantes como *Benign* o *SMS*. Este desequilibrio es un factor crítico para el diseño del modelo, ya que implica que el clasificador podría presentar un sesgo hacia las clases mayoritarias si no se aplican técnicas de compensación durante el entrenamiento.

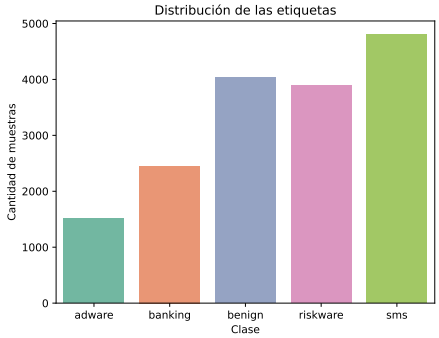

Figure 1: Distribución de etiquetas por tipo de Malware.

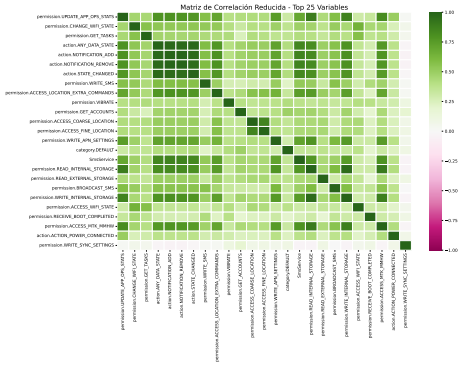

Figure 2: Matriz de correlación de las variables con mayor coeficiente de interacción.

Respecto al análisis de la matriz de correlación (Figura 2), se identificaron grupos de variables con una dependencia lineal casi perfecta. Un ejemplo destacado es la correlación entre *action.NOTIFICATION_ADD* y *action.NOTIFICATION_REMOVE*. Estas características se refieren a las peticiones del sistema para gestionar el ciclo de vida de las notificaciones en el dispositivo. La alta correlación se debe a que las aplicaciones que implementan servicios de escucha de notificaciones suelen requerir ambas acciones de manera simétrica para añadir o eliminar alertas de la interfaz, lo que genera redundancia en el espacio de características. Esto indica que es importante hacer una selección de características o reducción de dimensionalidad.

Para evaluar la separabilidad del dataset, se aplicó la técnica de reducción de dimensionalidad **t-SNE** (Figura 3). El resultado muestra que ciertas familias de malware, particularmente *Adware*, tienden a formar *clusters* o agrupamientos densos y definidos en el espacio reducido, lo que indica una firma conductual muy consistente. Por el contrario, la clase *Benign* aparece mucho más dispersa; esto se explica por la enorme heterogeneidad de las aplicaciones legítimas, las cuales no comparten un patrón funcional tan rígido como el software malicioso.

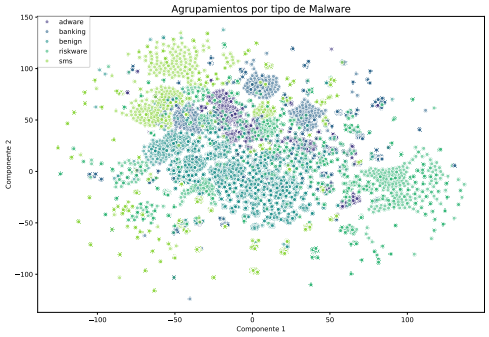

Figure 3: Proyección de alta dimensionalidad mediante t-SNE.

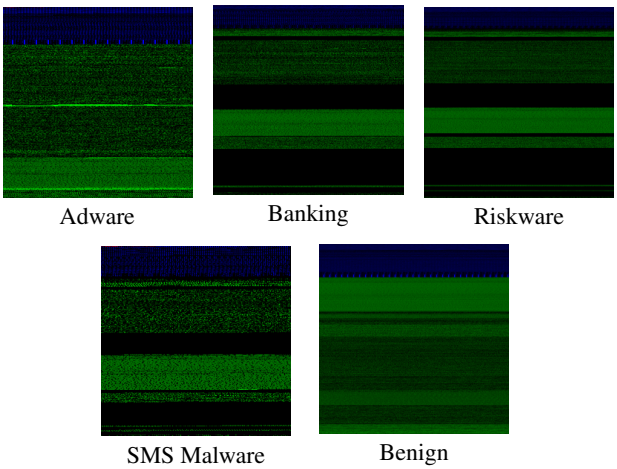

Figure 4: Ejemplos de representaciones binarias RGB para cinco categorías: Adware, Banking, Riskware, SMS Malware y Benign.

5.2 Imágenes

La modalidad visual del dataset consiste en imágenes de **256x256 píxeles** con tres canales de color (**RGB**). A un nivel puramente visual, la inspección de los binarios convertidos (Figura 4) revela que existen características morfológicas típicas en los archivos *.DEX*.

Se observa que las aplicaciones maliciosas suelen presentar patrones de textura, bloques de color y distribuciones de densidad de bytes que difieren de las aplicaciones benignas. Estas similitudes visuales dentro de una misma familia de malware sugieren que las Redes Neuronales Convolucionales (CNN) podrían extraer características espaciales relevantes que complementen la información lógica de los datos tabulares y textual de loas llamadas de API.

5.3 Secuencias

El dataset cuenta con 11,683 registros correspondientes a aplicaciones Android, incluyendo información de GML-names, hash del *APK*, la etiqueta de clase (*Label*), y la secuencia de llamadas a métodos de cada aplicación. Cada

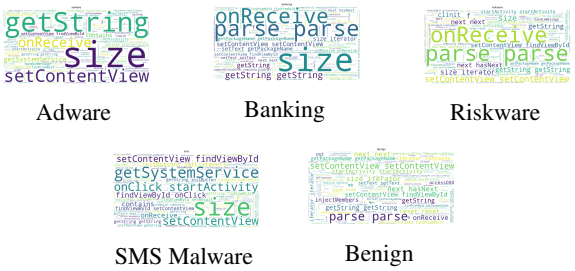

Figure 5: *Wordclouds* de secuencias de APIs por categoría.

secuencia de texto representa el orden de ejecución de las llamadas a métodos dentro del binario y captura patrones de comportamiento dinámico de la aplicación. La clase asociada indica la categoría a la que pertenece la muestra, permitiendo entrenar modelos de clasificación supervisada que identifiquen aplicaciones maliciosas frente a benignas.

La Figura 5 muestra los *wordclouds* construidos a partir de las secuencias de llamadas a APIs por categoría de malware. Así, por ejemplo en la categoría SMS se observan métodos frecuentes como `getImei`, `sendTextMessage` y `getSystemService`, indicando acceso a servicios de telefonía y envío de mensajes automáticos. También aparecen llamadas relacionadas con la interfaz de usuario, como `setContentView` y `onClick`, lo que sugiere que algunas aplicaciones maliciosas simulan interfaces legítimas. Estas diferencias en los patrones de llamadas entre categorías refuerzan la utilidad de los modelos secuenciales para capturar comportamientos específicos de cada familia de malware.

5.4 Repositorio

El repositorio en el que se está desarrollando el proyecto en GitHub: Repositorio MLT_project.

References

BankMyCell. How many android users are there?, 2026. URL `https://www.bankmycell.com/blog/how-many-android-users-are-there`. Accessed: 2026-03-14.

James Arrowsmith, Teo Susnjak, and Julian Jang-Jaccard. Multimodal deep learning for android malware classification. *Machine Learning and Knowledge Extraction*, 7 (1), 2025. ISSN 2504-4990. doi:10.3390/make7010023. URL `https://www.mdpi.com/2504-4990/7/1/23`.

Doan Minh Trung, Tien Duc Anh Hao, Luong Hoang Minh, Nghi Hoang Khoa, Nguyen Tan Cam, Van-Hau Pham, and Phan The Duy. Dmldroid: Deep multimodal fusion framework for android malware detection with resilience to code obfuscation and adversarial perturbations, 2025. URL `https://arxiv.org/abs/2509.11187`.

Zhiqiang Wang, Qiulong Yu, and Sicheng Yuan. Android malware detection based on rgb images and multi-

feature fusion, 2024. URL https://arxiv.org/abs/2408.16555.

Kaggle. Multimodal cic andmal2020, 2020. URL https://www.kaggle.com/datasets/haotienducanh/multimodal-cic-andmal2020. Accessed: 2026-03-01.

Samaneh Mahdavifar, Andi Fitriah Abdul Kadir, Rasool Fatemi, Dima Alhadidi, and Ali A. Ghorbani. Dynamic android malware category classification using semi-supervised deep learning. pages 515–522, 2020. doi:10.1109/DASC-PICom-CBDCom-CyberSciTech49142.2020.00094.

Samaneh Mahdavifar, Dima Alhadidi, and Ali. A. Ghorbani. Effective and efficient hybrid android malware classification using pseudo-label stacked auto-encoder. *Journal of Network and Systems Management*, 30(1):22, 2021. doi:10.1007/s10922-021-09634-4. URL https://doi.org/10.1007/s10922-021-09634-4.